



Lecture 13

Compiler Design CSC-305

Awanish Pandey

Department of Computer Science and Engineering
Indian Institute of Technology
Roorkee

August 18, 2026

Applying symbols in a state

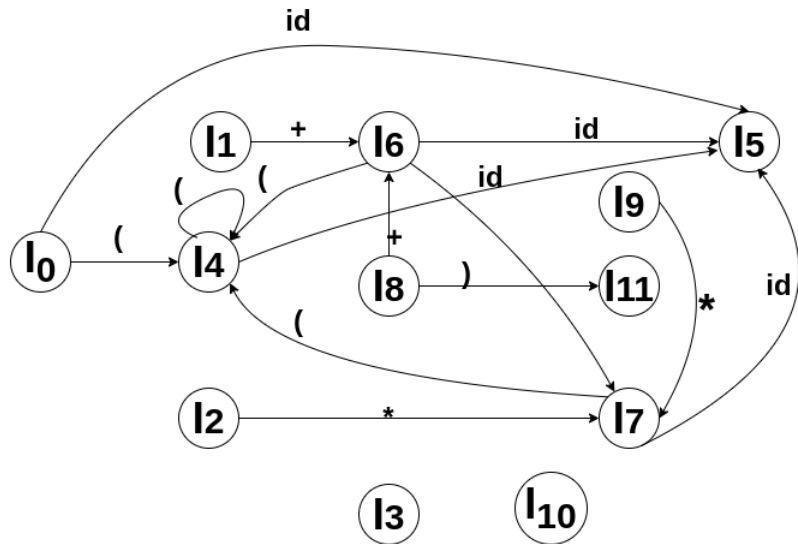
Applying symbols in a state

- Create new state and include all the items that have appropriate input symbol just after the "."

Applying symbols in a state

- Create new state and include all the items that have appropriate input symbol just after the "."
- Advance "." in those items and take closure

Example



Goto operation

Goto operation

- $Goto(I, X)$, where I is a set of items and X is a grammar symbol

Goto operation

- $Goto(I, X)$, where I is a set of items and X is a grammar symbol
 - ▶ is closure of set of item $A \rightarrow \alpha X.\beta$

Goto operation

- $Goto(I, X)$, where I is a set of items and X is a grammar symbol
 - ▶ is closure of set of item $A \rightarrow \alpha X.\beta$
 - ▶ such that $A \rightarrow \alpha.X\beta$ is in I

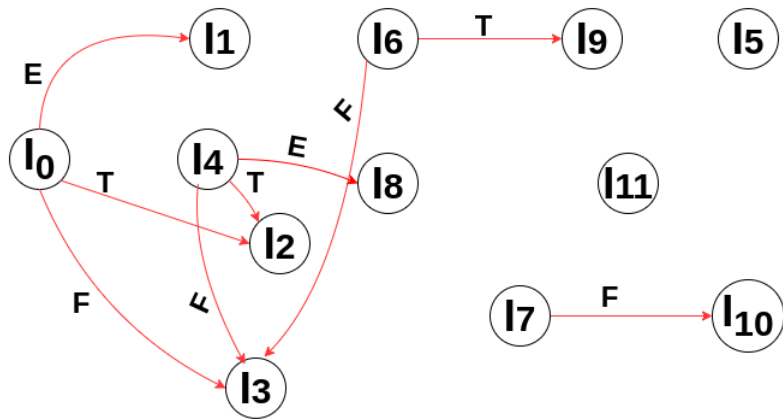
Goto operation

- $Goto(I, X)$, where I is a set of items and X is a grammar symbol
 - ▶ is closure of set of item $A \rightarrow \alpha X.\beta$
 - ▶ such that $A \rightarrow \alpha.X\beta$ is in I
- Intuitively if I is a set of items for some valid prefix α then $goto(I, X)$ is set of valid items for prefix αX

Goto operation

- $Goto(I, X)$, where I is a set of items and X is a grammar symbol
 - ▶ is closure of set of item $A \rightarrow \alpha X \beta$
 - ▶ such that $A \rightarrow \alpha.X\beta$ is in I
- Intuitively if I is a set of items for some valid prefix α then $goto(I, X)$ is set of valid items for prefix αX
- If I is $E' \rightarrow E$.
 $E \rightarrow E. + T$
then $goto(I, +)$ is
 $E \rightarrow E + .T$
 $T \rightarrow .T * F$
 $T \rightarrow .F$
 $F \rightarrow .(E)$
 $F \rightarrow .id$

Example



Construct SLR(Simple LR) parse table

- Construct $C = I_0, \dots, I_n$ the collection of sets of $LR(0)$ items.

Construct SLR(Simple LR) parse table

- Construct $C = I_0, \dots, I_n$ the collection of sets of $LR(0)$ items.
- If $A \rightarrow \alpha.a\beta$ is in I_i and $goto(I_i, a) = I_j$ then $action[i, a] = \text{shift } j$

Construct SLR(Simple LR) parse table

- Construct $C = I_0, \dots, I_n$ the collection of sets of $LR(0)$ items.
- If $A \rightarrow \alpha.a\beta$ is in I_i and $goto(I_i, a) = I_j$ then $action[i, a] = \text{shift } j$
- If $A \rightarrow \alpha.$ is in I_i
then $action[i, a] = \text{reduce } A \rightarrow \alpha$ for all a in $follow(A)$

Construct SLR(Simple LR) parse table

- Construct $C = I_0, \dots, I_n$ the collection of sets of $LR(0)$ items.
- If $A \rightarrow \alpha.a\beta$ is in I_i and $goto(I_i, a) = I_j$ then $action[i, a] = \text{shift } j$
- If $A \rightarrow \alpha.$ is in I_i
then $action[i, a] = \text{reduce } A \rightarrow \alpha$ for all a in $follow(A)$
- If $S' \rightarrow S.$ is in I_i then $action[i, \$] = \text{accept}$

Construct SLR(Simple LR) parse table

- Construct $C = I_0, \dots, I_n$ the collection of sets of $LR(0)$ items.
- If $A \rightarrow \alpha.a\beta$ is in I_i and $goto(I_i, a) = I_j$ then $action[i, a] = \text{shift } j$
- If $A \rightarrow \alpha.$ is in I_i
then $action[i, a] = \text{reduce } A \rightarrow \alpha$ for all a in $follow(A)$
- If $S' \rightarrow S.$ is in I_i then $action[i, \$] = \text{accept}$
- If $goto(I_i, A) = I_j$
then $goto[i, A] = j$ for all non terminals A

Construct SLR(Simple LR) parse table

- Construct $C = I_0, \dots, I_n$ the collection of sets of $LR(0)$ items.
- If $A \rightarrow \alpha.a\beta$ is in I_i and $goto(I_i, a) = I_j$ then $action[i, a] = \text{shift } j$
- If $A \rightarrow \alpha.$ is in I_i
then $action[i, a] = \text{reduce } A \rightarrow \alpha$ for all a in $follow(A)$
- If $S' \rightarrow S.$ is in I_i then $action[i, \$] = \text{accept}$
- If $goto(I_i, A) = I_j$
then $goto[i, A] = j$ for all non terminals A
- All entries not defined are errors

Construct SLR(Simple LR) parse table

- Construct $C = I_0, \dots, I_n$ the collection of sets of $LR(0)$ items.
- If $A \rightarrow \alpha.a\beta$ is in I_i and $goto(I_i, a) = I_j$ then $action[i, a] = \text{shift } j$
- If $A \rightarrow \alpha.$ is in I_i
then $action[i, a] = \text{reduce } A \rightarrow \alpha$ for all a in $follow(A)$
- If $S' \rightarrow S.$ is in I_i then $action[i, \$] = \text{accept}$
- If $goto(I_i, A) = I_j$
then $goto[i, A] = j$ for all non terminals A
- All entries not defined are errors

SLR is too weak to handle most languages!

Homework

- Create SLR Parse table for the following grammar (homework)

$$S' \rightarrow S$$

$$S \rightarrow L = R$$

$$S \rightarrow R$$

$$L \rightarrow *R$$

$$L \rightarrow id$$

$$R \rightarrow L$$

Parse Table

SLR parse table for the grammar

	=	*	id	\$	S	L	R
0		s4	s5		1	2	3
1				acc			
2	s6,r6			r6			
3				r3			
4		s4	s5			8	7
5	r5			r5			
6		s4	s5			8	9
7	r4			r4			
8	r6			r6			
9				r2			

The table has multiple entries in action[2,=]

Problems in SLR parsing

- No sentential form of this grammar can start with $R = \dots$

Problems in SLR parsing

- No sentential form of this grammar can start with $R = \dots$
- However, the reduce action in action[2,=] generates a sentential form starting with $R = \dots$

Problems in SLR parsing

- No sentential form of this grammar can start with $R = \dots$
- However, the reduce action in action[2,=] generates a sentential form starting with $R = \dots$
- Therefore, the reduce action is incorrect

Problems in SLR parsing

- No sentential form of this grammar can start with $R = \dots$
- However, the reduce action in action[2,=] generates a sentential form starting with $R = \dots$
- Therefore, the reduce action is incorrect
- In SLR parsing method state i calls for reduction on symbol "a", by rule $A \rightarrow \alpha$ if I_i contains $[A \rightarrow \alpha.]$ and "a" is in follow(A)

Problems in SLR parsing

- No sentential form of this grammar can start with $R = \dots$
- However, the reduce action in action[2,=] generates a sentential form starting with $R = \dots$
- Therefore, the reduce action is incorrect
- In SLR parsing method state i calls for reduction on symbol "a", by rule $A \rightarrow \alpha$ if I_i contains $[A \rightarrow \alpha.]$ and "a" is in follow(A)
- However, when state I appears on the top of the stack, the viable prefix $\beta\alpha$ on the stack may be such that $\beta\alpha$ can not be followed by symbol "a" in any right sentential form

Problems in SLR parsing

- No sentential form of this grammar can start with $R = \dots$
- However, the reduce action in action[2,=] generates a sentential form starting with $R = \dots$
- Therefore, the reduce action is incorrect
- In SLR parsing method state i calls for reduction on symbol "a", by rule $A \rightarrow \alpha$ if I_i contains $[A \rightarrow \alpha.]$ and "a" is in follow(A)
- However, when state I appears on the top of the stack, the viable prefix $\beta\alpha$ on the stack may be such that $\beta\alpha$ can not be followed by symbol "a" in any right sentential form
- Thus, the reduction by the rule $A \rightarrow \alpha$ on symbol "a" is invalid

Problems in SLR parsing

- No sentential form of this grammar can start with $R = \dots$
- However, the reduce action in action[2,=] generates a sentential form starting with $R = \dots$
- Therefore, the reduce action is incorrect
- In SLR parsing method state i calls for reduction on symbol "a", by rule $A \rightarrow \alpha$ if I_i contains $[A \rightarrow \alpha.]$ and "a" is in follow(A)
- However, when state I appears on the top of the stack, the viable prefix $\beta\alpha$ on the stack may be such that $\beta\alpha$ can not be followed by symbol "a" in any right sentential form
- Thus, the reduction by the rule $A \rightarrow \alpha$ on symbol "a" is invalid
- SLR parsers cannot remember the left context

Canonical LR Parsing

- Carry extra information in the state so that wrong reductions by $A \rightarrow \alpha$ will be ruled out

Canonical LR Parsing

- Carry extra information in the state so that wrong reductions by $A \rightarrow \alpha$ will be ruled out
- Redefine LR items to include a terminal symbol as a second component (look ahead symbol).

Canonical LR Parsing

- Carry extra information in the state so that wrong reductions by $A \rightarrow \alpha$ will be ruled out
- Redefine LR items to include a terminal symbol as a second component (look ahead symbol).
- The general form of the item becomes $[A \rightarrow \alpha.\beta, a]$ which is called LR(1) item.

Canonical LR Parsing

- Carry extra information in the state so that wrong reductions by $A \rightarrow \alpha$ will be ruled out
- Redefine LR items to include a terminal symbol as a second component (look ahead symbol).
- The general form of the item becomes $[A \rightarrow \alpha.\beta, a]$ which is called LR(1) item.
- Item $[A \rightarrow \alpha., a]$ calls for reduction only if next input is a . The set of symbols " a "s will be a subset of $Follow(A)$

Closure(I)

```
repeat
  for each item  $[A \rightarrow \alpha.B\beta, a]$  in I
    for each production  $B \rightarrow \gamma$  in  $G'$ 
      and for each terminal  $b$  in  $First(\beta a)$ 
        add item  $[B \rightarrow .\gamma, b]$  to I
until no more additions to I
```

Example

- Consider the following grammar

$$S' \rightarrow S$$

$$S \rightarrow CC$$

$$C \rightarrow cC \mid d$$

- Compute closure(I) where $I = [S' \rightarrow .S, \$]$

$$S \rightarrow .S, \$$$

$$S \rightarrow .CC, \$$$

$$C \rightarrow .cC, c$$

$$C \rightarrow .cC, d$$

$$C \rightarrow .d, c$$

$$C \rightarrow .d, d$$

Example

Construct sets of LR(1) items for the grammar on previous slide

- $I_0 : S \rightarrow .S, \$$
 $S \rightarrow .CC, \$$
 $C \rightarrow .cC, c/d$
 $C \rightarrow .d, c/d$
- $I_1 : \text{goto}(I_0, S)$
 $S' \rightarrow S., \$$
- $I_2 : \text{goto}(I_0, C)$
 $S \rightarrow C.C, \$$
 $C \rightarrow .cC, \$$
 $C \rightarrow .d, \$$
- $I_3 : \text{goto}(I_0, c)$
 $C \rightarrow c.C, c/d$
 $C \rightarrow .cC, c/d$
 $C \rightarrow .d, c/d$
- $I_4 : \text{goto}(I_0, d)$
 $C \rightarrow d., c/d$
- $I_5 : \text{goto}(I_2, C)$
 $S \rightarrow CC., \$$
- $I_6 : \text{goto}(I_2, c)$
 $C \rightarrow c.C, \$$
 $C \rightarrow .cC, \$$
 $C \rightarrow .d, \$$
- $I_7 : \text{goto}(I_2, d)$
 $C \rightarrow d., \$$
- $I_8 : \text{goto}(I_3, C)$
 $C \rightarrow cC., c/d$
- $I_9 : \text{goto}(I_6, C)$
 $C \rightarrow cC., \$$

Construction of Canonical LR parse table

- Construct $C = I_0, \dots, I_n$ the sets of LR(1) items

Construction of Canonical LR parse table

- Construct $C = I_0, \dots, I_n$ the sets of LR(1) items
- If $[A \rightarrow \alpha.a\beta, \quad b]$ is in I_i and $goto(I_i, a) = I_j$ then $action[i, a] = \text{shift } j$

Construction of Canonical LR parse table

- Construct $C = I_0, \dots, I_n$ the sets of LR(1) items
- If $[A \rightarrow \alpha.a\beta, \quad b]$ is in I_i and $goto(I_i, a) = I_j$ then $action[i, a] = \text{shift } j$
- If $[A \rightarrow \alpha., a]$ is in I_i then $action[i, a]$ reduce $A \rightarrow \alpha$

Construction of Canonical LR parse table

- Construct $C = I_0, \dots, I_n$ the sets of LR(1) items
- If $[A \rightarrow \alpha.a\beta, \quad b]$ is in I_i and $goto(I_i, a) = I_j$ then $action[i, a] = \text{shift } j$
- If $[A \rightarrow \alpha., a]$ is in I_i then $action[i, a] = \text{reduce } A \rightarrow \alpha$
- If $[S' \rightarrow S., \$]$ is in I_i then $action[i, \$] = \text{accept}$

Construction of Canonical LR parse table

- Construct $C = I_0, \dots, I_n$ the sets of LR(1) items
- If $[A \rightarrow \alpha.a\beta, \quad b]$ is in I_i and $\text{goto}(I_i, a) = I_j$ then $\text{action}[i, a] = \text{shift } j$
- If $[A \rightarrow \alpha., a]$ is in I_i then $\text{action}[i, a] = \text{reduce } A \rightarrow \alpha$
- If $[S' \rightarrow S., \$]$ is in I_i then $\text{action}[i, \$] = \text{accept}$
- If $\text{goto}(I_i, A) = I_j$ then $\text{goto}[i, A] = j$ for all non terminals A

Parse table

State	c	d	\$	S	C
0	s3	s4		1	2
1			acc		
2	s6	s7			5
3	s3	s4			8
4	r3	r3			
5			r1		
6	s6	s7			9
7			r3		
8	r2	r2			
9			r2		